Module 0

Installation and Setup

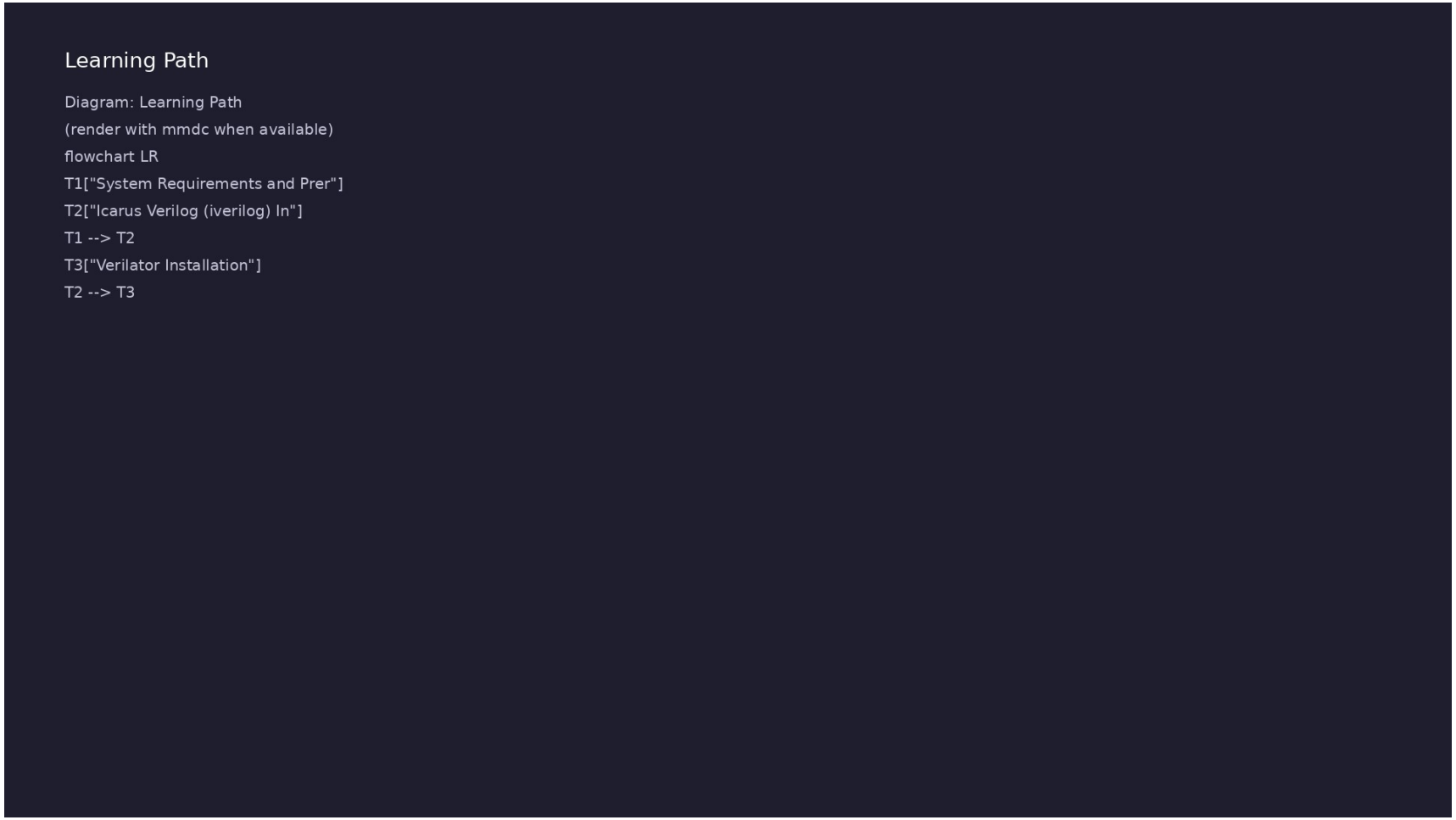
Learning objectives

- Tool Installation: Automated and manual installation methods for all verification tools
- Environment Setup: Configure your development environment for efficient verification work
- First Testbenches: Create and run your first "Hello World" testbenches in both paradigms
- Waveform Analysis: Generate and view waveforms using GTKWave
- Project Structure: Organize your verification project following best practices
- Tool Comparison: Understand when to use iverilog vs Verilator

Prerequisites

- Have a Linux, macOS, or Windows (WSL2) system
- Have basic command-line knowledge
- Have Git installed (for cloning the repository)
- Have a C/C++ compiler available (GCC, Clang, or MSVC)

Learning path



Set up verification environment with iverilog and Verilator

Overview

- This module covers the complete setup of your verification environment, including installation of i...

Design architecture

1. Verification toolchain layout (1/2)

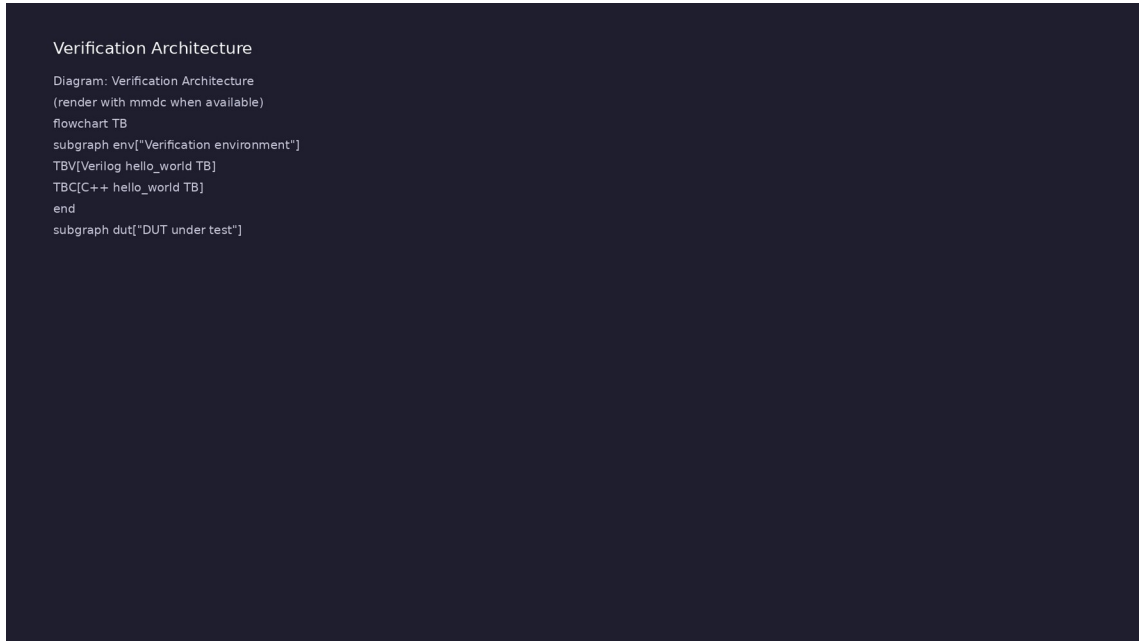
- Host environment: Linux, macOS, or WSL2 with GCC/Clang, Make, and Git
- Icarus path: `iverilog` compiles RTL + TB → `simv` (or named binary) executed by `vvp`
- Verilator path: `verilator --cc` emits C++ model under `obj_dir/` linked with a C++ testbench
- Waveform path: Simulators write VCD/FST; GTKWave visualizes signal transitions

1. Verification toolchain layout (2/2)

- Course layout: `scripts/install_*.sh`,
  `module0/examples/`, `module0/dut/`,
  `module0/tests/`

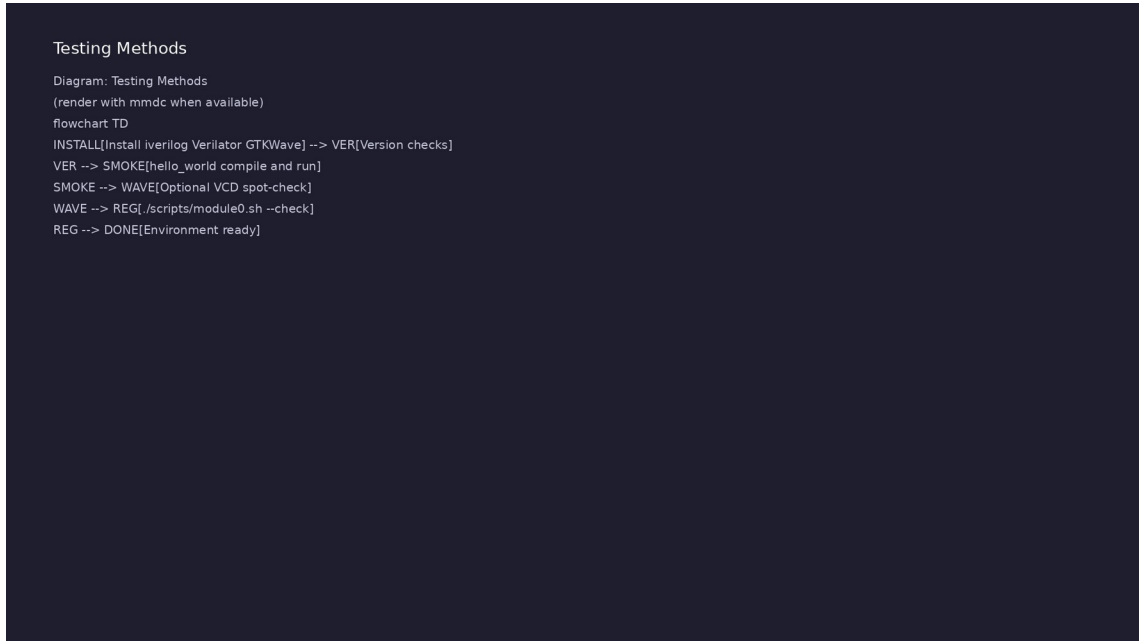
2. Reference DUT hierarchy (Module 0)

- `module0/dut/simple_gates/`: Combinational `and_gate`, `or_gate` — minimal port lists (a, b, y)
- `module0/dut/counters/`: `simple_counter` — clocked sequential logic with enable and reset
- Hierarchy depth: Flat RTL (no sub-modules) so install smoke tests stay fast
- TB attachment: Testbench modules instantiate DUT at top level; no bus fabric yet



3. First simulation artifact flow

- iverilog: `iverilog -o sim tb.v dut.v` → `vvp sim` → optional `$dumpfile` VCD
- Verilator: `verilator --cc --exe` → `make -C obj_dir` → `./obj_dir/Vdut`
- Build dirs: Per-example `Makefile` targets; artifacts under `build/` or `obj_dir/` (gitignored)



RTL block diagram (reference)

Module 0: DUT hierarchy and signal flow.

iverilog — compile then simulate

```
initial begin
        // $display is a system task that prints formatted text to
stdout
        // Similar to printf() in C or print() in Python
        $display("=====================================");
        $display("Hello World from Icarus Verilog!");
        $display("=====================================");
        $display("If you see this message, iverilog is working
correctly.");

        // $time is a system function that returns current simulation
time
        // %0t formats it as a decimal number (no leading zeros)
        $display("Time: %0t", $time);
        $display("=====================================");

        // #10 introduces a delay of 10 time units
```

Verilator — C++ model and eval()

```
Verilated::commandArgs(argc, argv);

    // Print test header
    std::cout << "========================================" <<
std::endl;
    std::cout << "Hello World from Verilator!" << std::endl;
    std::cout << "========================================" <<
std::endl;
    std::cout << "If you see this message, Verilator is working
correctly." << std::endl;

    /**
     * Create DUT (Design Under Test) Instance
     *
     * Verilator generates a C++ class for each Verilog module.
     * Class naming convention: V<module_name>
     *
```

Reference DUT — and_gate

```
always @(*) begin
        // AND operation: y is 1 only when both a and b are 1
        // This is the fundamental AND gate behavior
        y = a & b;
    end

endmodule
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until $finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module0/examples/<example>/ — run make run from that directory

Directed test execution sequence (1)

- make hello_world
- PASS console message
- optional GTKWave on .vcd
- Install tools
- iverilog -v / verilator --version

Directed test execution sequence (2)

- iverilog -o sim tb.v
- vvp sim
- optional $dumpfile VCD

Makefile — iverilog hello_world target

```
hello_world: hello_world.v
	@echo "Compiling hello_world..."
	# Compile Verilog file into executable
	# -o: output filename
	$(IVERILOG) -o hello_world hello_world.v
	@echo "Running hello_world..."
	# Run compiled simulation
	$(VVP) hello_world
```

Verilator Makefile — build and run

```
hello_world: hello_world.v hello_world.cpp
        @echo "Compiling hello_world with Verilator..."
        # Verilator generates C++ class from Verilog module
        # Then compiles C++ testbench with generated code
        # Creates executable: ./obj_dir/Vhello_world
        $(VERILATOR) $(VERILATOR_FLAGS) hello_world.v hello_world.cpp
        @echo "Running hello_world..."
        # Execute the compiled simulation
        ./obj_dir/Vhello_world
```

GTKWave example — VCD generation

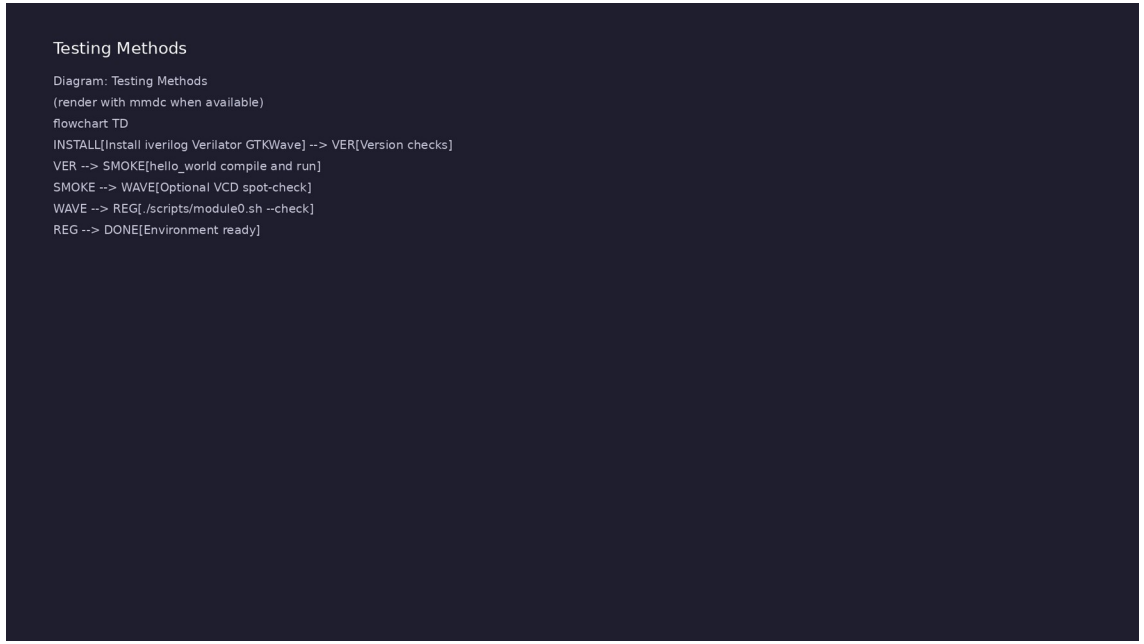
```
/**
 * GTKWave Usage Example for iverilog
 *
 * This testbench demonstrates:
 * - VCD file generation
 * - Multiple signal types (clock, data, control)
 * - Signal transitions for waveform analysis
 * - GTKWave viewing workflow
 *
 * Usage:
 *   iverilog -o gtkwave_example gtkwave_example.v
../../dut/simple_gates/and_gate.v
 *   vvp gtkwave_example
 *   gtkwave gtkwave_example.vcd
 */
```

```
`timescale 1ns/1ps
```

Verification & testing methods

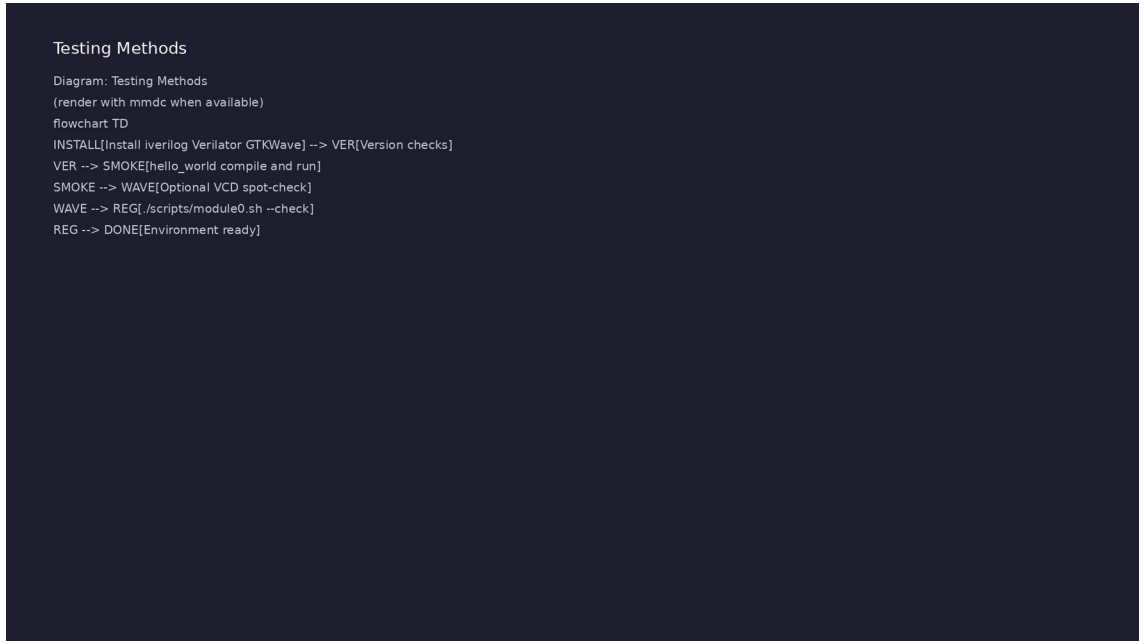
1. Install and environment verification

- Version checks: `iverilog -v`, `verilator --version`, `gtkwave --version`
- PATH and compiler: Confirm `g++`/`gcc` available for Verilator link step
- Self-check: `./scripts/module0.sh --check` validates examples and DUT compiles



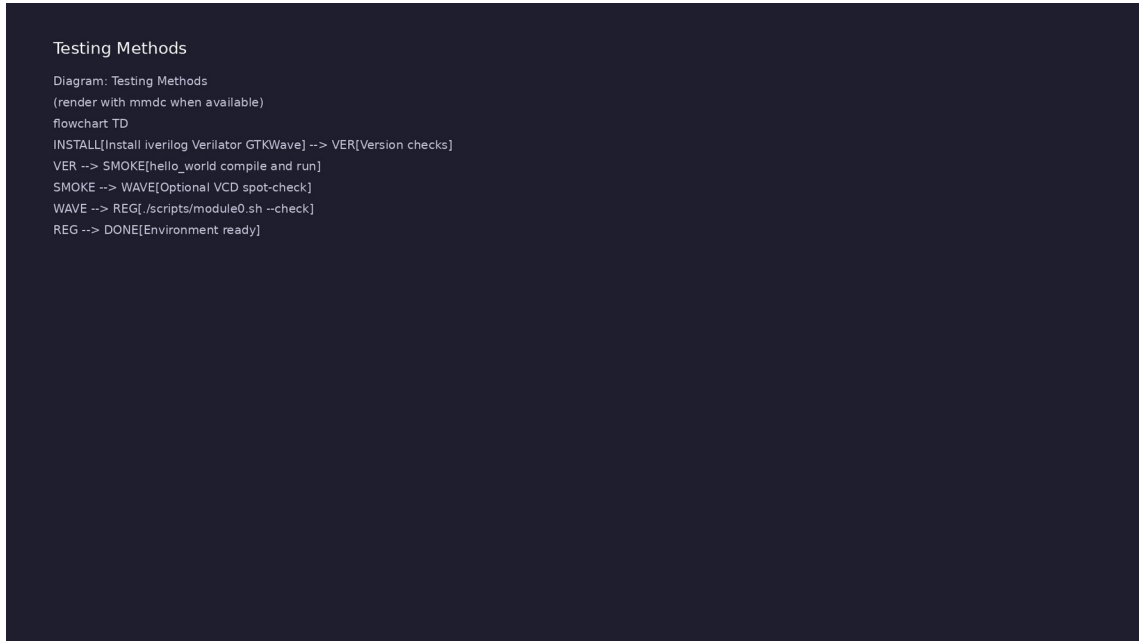
2. Hello-world test methodology

- Smoke test: Minimal compile-and-run per tool (no functional depth required)
- Pass criteria: Clean compile, simulation exits 0, expected `$display` or C++ `printf`
- Waveform spot-check: Open generated VCD in GTKWave to confirm clock/reset toggles



3. Debugging and regression basics

- Log-first: Capture simulator stdout; use `-g` / `--trace` when examples enable it
- Compare tools: Run same DUT with iverilog vs Verilator to see flow differences
- Repeatability: Document working directory and Make target in `commands_to_try.txt`



Install verification — version checks

```
#!/bin/bash

# Icarus Verilog (iverilog) Installation Script
# Installs iverilog from system package manager or builds from source
# Usage: ./install_iverilog.sh [--system] [--source] [OPTIONS]

set -euo pipefail

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

# Get script directory
# ...
```

Syllabus topics

Protocol & design details

Detail: iverilog Installation Issues (1)

- Cause: iverilog not installed or not in PATH
- Solution:
- Verify: `iverilog -v` should show version
- Cause: SystemVerilog syntax not supported by default

Detail: iverilog Installation Issues (2)

- Solution: Use `-g2012` flag for SystemVerilog support
- Note: Some SystemVerilog features may still be limited
- Cause: Missing include path
- Solution: Add include directory with `-I` flag

Detail: Verilator Installation Issues (1)

- Cause: Verilator not installed or not in PATH
- Solution:
- Cause: Installed version is too old
- Solution: Install from source for latest version

Detail: Verilator Installation Issues (2)

- Cause: C++ compiler not installed
- Solution: Install build essentials
- Cause: Verilator headers not found
- Solution: Check Verilator installation and include paths

Detail: GTKWave Issues (1)

- Cause: GTKWave not installed
- Solution:
- Cause: File format issue or corrupted file
- Solution:

Detail: GTKWave Issues (2)

- Verify VCD file was generated correctly
- Check file permissions
- Try regenerating: `vvp executable` (should create .vcd file)

Detail: Platform-Specific Issues (1)

- Issue: Package manager version may be outdated
- Solution: Use installation scripts or build from source
- Issue: Homebrew version may be outdated
- Solution: Use `--source` flag to build from source

Detail: Platform-Specific Issues (2)

- Issue: Some tools may not work in Windows directly
- Solution: Use WSL2 Ubuntu and install there

Detail: Build System Issues

- Issue: `make: *** No targets specified and no makefile found`
- Solution: Ensure you're in the correct directory with a Makefile
- Issue: Files not found during compilation
- Solution: Check relative paths and use `-I` flags correctly

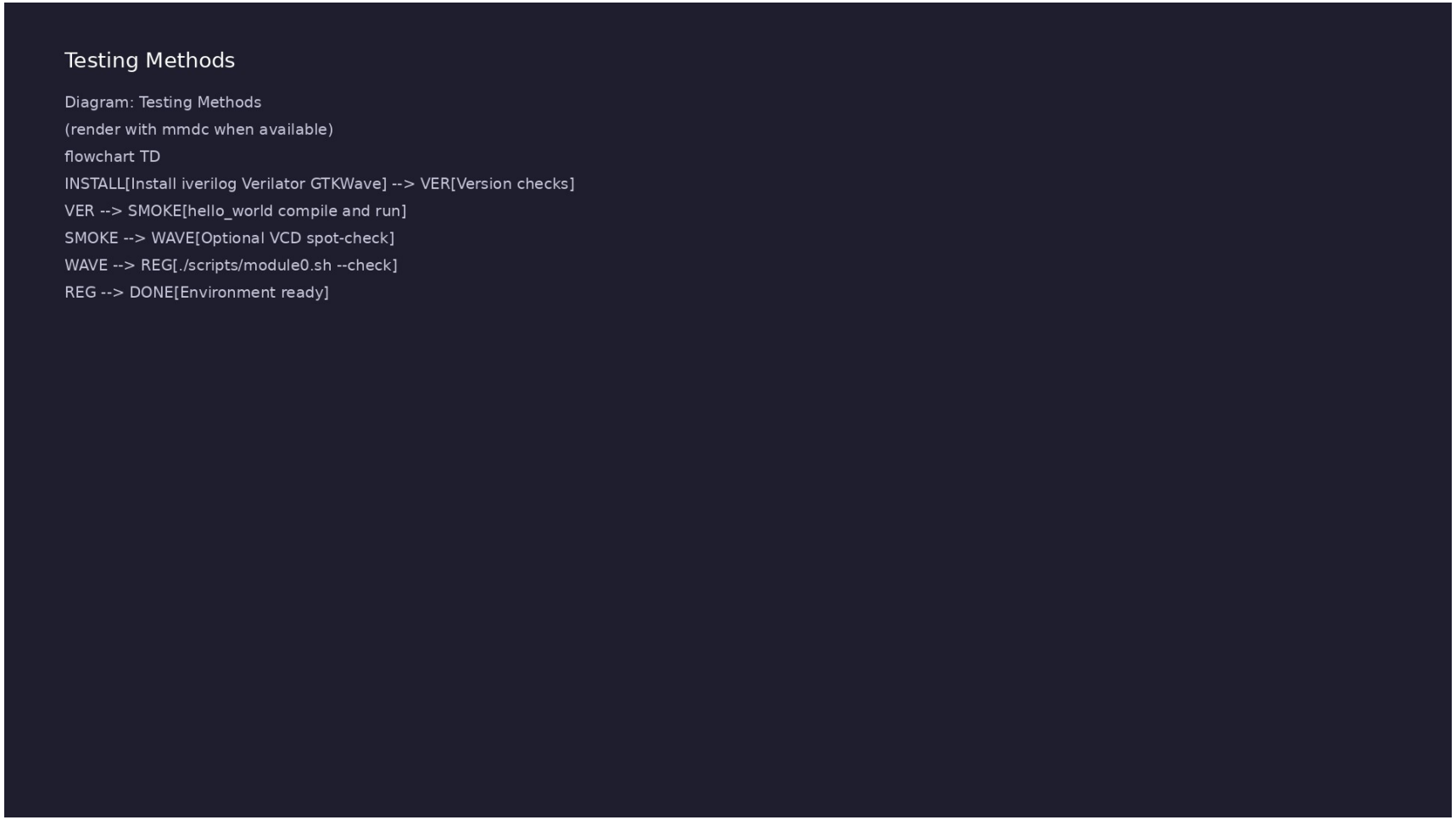
Detail: IDE Issues (1)

- VS Code: Install "SystemVerilog" extension by mshr-h
- Vim: Install vim-systemverilog plugin
- Verification: Open a `.sv` file and check for syntax highlighting
- Issue: IDE can't find included files

Detail: IDE Issues (2)

- Solution: Configure include paths in IDE settings
- VS Code: Add to `settings.json`:
- Issue: Can't set breakpoints or debug
- Solution:

Verification flow (reference)



Stimulus, DUT response, and check points for this module.

1. System Requirements and Prerequisites (1/3)

- Operating System Support
- Linux (Ubuntu/Debian, CentOS/RHEL, Fedora)
- macOS (Intel and Apple Silicon)
- Windows (WSL2 recommended)
- Hardware Requirements
- Minimum 4GB RAM (8GB+ recommended)

1. System Requirements and Prerequisites (2/3)

- 5GB free disk space
- Multi-core processor recommended
- Software Prerequisites
- C/C++ compiler (GCC 7+, Clang 10+, or MSVC)
- Make or Ninja build system
- Git

1. System Requirements and Prerequisites (3/3)

- Perl (for some tools)

2. Icarus Verilog (iverilog) Installation (1/5)

- What is iverilog?
- Open-source Verilog/SystemVerilog simulator
- Good for learning and prototyping
- Supports most Verilog constructs
- Limited SystemVerilog support
- Automated Installation (Recommended)

2. Icarus Verilog (iverilog) Installation (2/5)

- Using the installation script:
- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Builds and installs iverilog
- Verifies the installation

2. Icarus Verilog (iverilog) Installation (3/5)

- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: `sudo apt-get install iverilog gtkwave`
- CentOS/RHEL: `sudo yum install iverilog gtkwave`
- Fedora: `sudo dnf install iverilog gtkwave`
- Building from source (latest features)

2. Icarus Verilog (iverilog) Installation (4/5)

- macOS Installation
- Homebrew installation: `brew install icarus-verilog gtkwave`
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: `sudo apt-get install iverilog gtkwave`
- Building from source in WSL2

2. Icarus Verilog (iverilog) Installation (5/5)

- Verification Steps
- Check iverilog version: `iverilog -v`
- Check vvp runtime: `vvp -v`
- Run simple test: `echo 'module test; initial
  $display("Hello"); endmodule' | iverilog -o test -
  &&...

3. Verilator Installation (1/5)

- What is Verilator?
- Fast Verilog/SystemVerilog simulator
- Generates C++ code from Verilog
- Excellent performance
- Limited SystemVerilog support (some features)
- Automated Installation (Recommended)

3. Verilator Installation (2/5)

- Using the installation script:
- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Sets up git submodule in `tools/verilator/`
- Builds and installs Verilator

3. Verilator Installation (3/5)

- Verifies the installation
- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: `sudo apt-get install verilator`
- CentOS/RHEL: `sudo yum install verilator`
- Fedora: `sudo dnf install verilator`

3. Verilator Installation (4/5)

- Building from source (latest features)
- macOS Installation
- Homebrew installation: `brew install verilator`
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: `sudo apt-get install verilator`

3. Verilator Installation (5/5)

- Building from source in WSL2
- Verification Steps
- Check Verilator version: `verilator --version`
- Run simple test: Create a minimal Verilog file and compile with Verilator

4. GTKWave Installation (1/2)

- What is GTKWave?
- Waveform viewer for VCD and FST files
- Essential for debugging testbenches
- Open-source and cross-platform
- Installation
- Usually installed with iverilog package

4. GTKWave Installation (2/2)

- Linux: `sudo apt-get install gtkwave` (or included with iverilog)
- macOS: `brew install gtkwave` (or included with icarus-verilog)
- Windows: Available as standalone installer
- Verification Steps
- Check GTKWave: `gtkwave --version`
- Open a VCD file: `gtkwave waveform.vcd`

5. Build System Configuration (1/4)

- Makefile Setup
- Basic Makefile for iverilog
- Basic Makefile for Verilator
- Compilation flags
- Include paths
- Test execution

5. Build System Configuration (2/4)

- iverilog Compilation
- Compilation command: `iverilog -o output input.v`
- Include paths: `-I<directory>`
- Define macros: `-D<macro>`
- Timescale handling
- Multi-file compilation

5. Build System Configuration (3/4)

- Verilator Compilation
- Compilation command: `verilator --cc --exe design.v testbench.cpp`
- SystemVerilog flags: `-sv`, `--timing`
- Include paths: `-I<directory>`
- Optimization flags: `-O0`, `-O1`, `-O2`, `-O3`
- Coverage options: `--coverage`

5. Build System Configuration (4/4)

- Waveform generation: `--trace`, `--trace-fst`

6. IDE Setup and Configuration (1/3)

- Recommended IDEs
- VS Code with Verilog/SystemVerilog extension
- Verible (SystemVerilog linter/formatter)
- Vim/Neovim with LSP
- Emacs with Verilog mode
- VS Code Configuration

6. IDE Setup and Configuration (2/3)

- SystemVerilog extension setup
- iverilog integration
- Verilator integration
- Debugging configuration
- Task runner setup for simulations
- Extension recommendations

6. IDE Setup and Configuration (3/3)

- Editor Configuration
- SystemVerilog formatting (Verible)
- Linting (Verilator, Verible)
- Code formatting on save
- Syntax highlighting

7. Project Structure Setup (1/4)

- Directory Structure
- Source code organization
- Test directory structure
- DUT (Design Under Test) organization
- Configuration files
- `tools/` directory for tools (Verilator as git submodule)

7. Project Structure Setup (2/4)

- Git Submodules Management
- Verilator is managed as a git submodule in `tools/verilator/`
- Initialize submodules: `./scripts/init_submodules.sh` or `git submodule update --init --recursive`
- Update submodules: `./scripts/update_submodules.sh` or `git submodule update --remote`
- Add new submodule: `./scripts/add_submodule.sh <repo_url> <path>`
- Remove submodule: `./scripts/remove_submodule.sh <path>`

7. Project Structure Setup (3/4)

- Makefile/Configuration
- Simple Makefile for running tests
- iverilog configuration
- Verilator configuration
- Environment variable management
- Version Control

7. Project Structure Setup (4/4)

- Git initialization
- .gitignore for Verilog and simulation (include build artifacts, waveforms)
- Initial commit structure
- Git submodules in `.gitmodules` file

8. First "Hello World" Verification Test (1/8)

- Prerequisites
- Ensure all tools are installed
- Verify tools are accessible: `iverilog -v` and
  `verilator --version`
- iverilog Hello World
- Location:
  `module0/examples/iverilog_basics/hello_world.v`
- Purpose: Simplest possible testbench to verify
  iverilog installation

8. First "Hello World" Verification Test (2/8)

- Key Concepts:
- `module` declaration (testbench has no ports)
- `initial` block for procedural code execution
- `$display` system task for output
- `$finish` system task to end simulation
- Time delays with `#delay`

8. First "Hello World" Verification Test (3/8)

- Compilation: `iverilog -o hello_world hello_world.v`
- Simulation: `vvp hello_world`
- Understanding Output: Console messages confirm successful execution
- GTKWave Waveform Example
- Location: `module0/examples/iverilog_basics/gtkwave_example.v`
- Purpose: Comprehensive example demonstrating waveform generation and analysis

8. First "Hello World" Verification Test (4/8)

- Key Concepts:
- VCD (Value Change Dump) file generation with `$dumpfile` and `$dumpvars`
- Clock generation using `forever` loops
- Multiple signal types (clock, data, control signals)
- Signal transitions for timing analysis
- Test phases and patterns

8. First "Hello World" Verification Test (5/8)

- Compilation: `iverilog -o gtkwave_example gtkwave_example.v ../../dut/simple_gates/and_gate.v`
- Simulation: `vvp gtkwave_example` (generates `gtkwave_example.vcd`)
- Viewing: `gtkwave gtkwave_example.vcd` or `make view_waveforms`
- GTKWave Features Demonstrated:
- Signal hierarchy navigation
- Time marker placement

8. First "Hello World" Verification Test (6/8)

- Zoom and pan operations
- Save file creation for quick reload
- Verilator Hello World
- Location:
  `module0/examples/verilator_basics/hello_world.cpp`
  and `hello_world.v`
- Purpose: Simplest possible C++ testbench to verify
  Verilator installation
- Key Concepts:

8. First "Hello World" Verification Test (7/8)

- Verilator compilation process (`verilator --cc --exe --build`)
- Generated C++ class structure (`V<module_name>`)
- DUT instantiation in C++
- `eval()` method for simulation
- `final()` method for cleanup
- Verilator initialization with `Verilated::commandArgs()`

8. First "Hello World" Verification Test (8/8)

- Compilation: `verilator --cc --exe --build hello_world.v hello_world.cpp`
- Running: `./obj_dir/Vhello_world`
- Understanding Output: Console messages confirm successful execution

9. Tool Comparison and Selection Criteria (1/4)

- iverilog Characteristics
- Verilog/SystemVerilog testbenches
- Good for learning
- Easy to use
- Good waveform support
- Moderate performance

9. Tool Comparison and Selection Criteria (2/4)

- Verilator Characteristics
- C++ testbenches
- Excellent performance
- Good for large designs
- Requires C++ knowledge
- Limited SystemVerilog support

9. Tool Comparison and Selection Criteria (3/4)

- When to Use Each
- Use iverilog for:
- Learning Verilog testbench concepts
- Quick prototyping
- Simple to medium complexity designs
- When you prefer Verilog syntax

9. Tool Comparison and Selection Criteria (4/4)

- Use Verilator for:
- High-performance simulation
- Large designs
- Integration with C++ libraries
- Complex testbenches with advanced data structures

10. Common Pitfalls and Solutions

- Pitfall 1: Installation Script Permissions
- Pitfall 2: Missing Build Dependencies
- Pitfall 3: Verilator Version Too Old
- Pitfall 4: PATH Not Updated

11. Troubleshooting Common Issues (1/8)

- Cause: iverilog not installed or not in PATH
- Solution:
- Verify: `iverilog -v` should show version
- Cause: SystemVerilog syntax not supported by default
- Solution: Use `-g2012` flag for SystemVerilog support
- Note: Some SystemVerilog features may still be limited

11. Troubleshooting Common Issues (2/8)

- Cause: Missing include path
- Solution: Add include directory with `-I` flag
- Cause: VVP runtime not installed or not in PATH
- Solution: VVP is usually installed with iverilog.

  Reinstall if missing:
- Cause: Verilator not installed or not in PATH
- Solution:

11. Troubleshooting Common Issues (3/8)

- Cause: Installed version is too old
- Solution: Install from source for latest version
- Cause: C++ compiler not installed
- Solution: Install build essentials
- Cause: Verilator headers not found
- Solution: Check Verilator installation and include paths

11. Troubleshooting Common Issues (4/8)

- Cause: Missing Verilator library linking
- Solution: Ensure Makefile includes Verilator libraries
- Cause: GTKWave not installed
- Solution:
- Cause: File format issue or corrupted file
- Solution:

11. Troubleshooting Common Issues (5/8)

- Verify VCD file was generated correctly
- Check file permissions
- Try regenerating: `vvp executable` (should create .vcd file)
- Issue: Package manager version may be outdated
- Solution: Use installation scripts or build from source
- Issue: Homebrew version may be outdated

11. Troubleshooting Common Issues (6/8)

- Solution: Use `--source` flag to build from source
- Issue: Some tools may not work in Windows directly
- Solution: Use WSL2 Ubuntu and install there
- Issue: `make: *** No targets specified and no makefile found`
- Solution: Ensure you're in the correct directory with a Makefile
- Issue: Files not found during compilation

11. Troubleshooting Common Issues (7/8)

- Solution: Check relative paths and use `-I` flags correctly
- VS Code: Install "SystemVerilog" extension by mshr-h
- Vim: Install vim-systemverilog plugin
- Verification: Open a `.sv` file and check for syntax highlighting
- Issue: IDE can't find included files
- Solution: Configure include paths in IDE settings

11. Troubleshooting Common Issues (8/8)

- VS Code: Add to `settings.json`:
- Issue: Can't set breakpoints or debug
- Solution:
- For Verilator: Use GDB with compiled executable
- For iverilog: Use VCD waveforms for debugging

12. Verification Checklist (1/3)

- C/C++ compiler installed and working
- iverilog installed and verified
- Using script: `./scripts/install_iverilog.sh`
- Verify: `iverilog -v` and `vvp -v`
- Verilator installed and verified
- Using script: `./scripts/install_verilator.sh`

12. Verification Checklist (2/3)

- Verify: `verilator --version`
- GTKWave installed and verified
- Verify: `gtkwave --version`
- All tools verified together
- IDE configured and working
- First test runs successfully (both iverilog and Verilator)

12. Verification Checklist (3/3)

- Can create and run simple testbench
- Understand basic project structure
- Know how to get help when stuck
- Understand tool differences and when to use each

Hands-on examples

Module 0 self-check

```
$ ./scripts/module0.sh --help
```

Terminal: module0_check

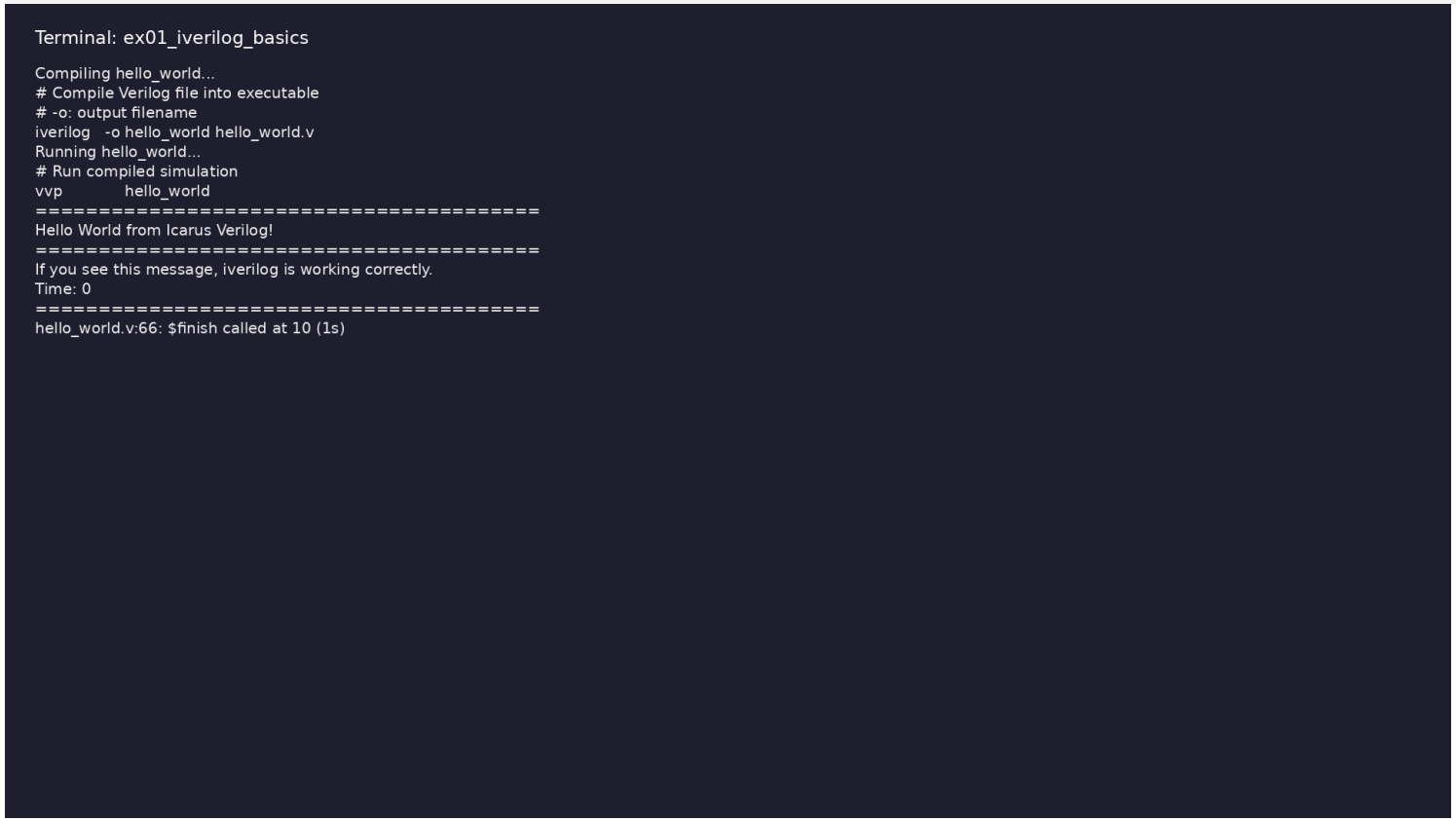
```
Usage: ./scripts/module0.sh [OPTIONS]

Module 0: Installation and Setup
This script runs examples and tests for Module 0.

OPTIONS:
  Examples:
    --iverilog-basics      Run iverilog basic examples
    --verilator-basics     Run Verilator basic examples
    --gtkwave-example      Run GTKWave waveform example (requires GUI)
    --all-examples         Run all examples (default)
    --skip-examples        Skip all examples

  Tests:
    --iverilog-tests       Run iverilog testbenches
    --verilator-tests      Run Verilator testbenches
    --all-tests            Run all tests (default)
    --skip-tests           Skip all tests

  Comparison:
    --comparison           Run comparison examples (default)
    --skip-comparison      Skip comparison examples

  Environment:
    --jobs N               Number of parallel build jobs (default: 8)
    --no-clean             Skip cleaning before build (faster rebuilds)

  Other:
    --help, -h             Show this help message

EXAMPLES:
  # Run all examples and tests
  ./scripts/module0.sh

  # Run only iverilog examples
  ./scripts/module0.sh --iverilog-basics
```

Exercise scaffold

```
./scripts/module0.sh --scaffold
```

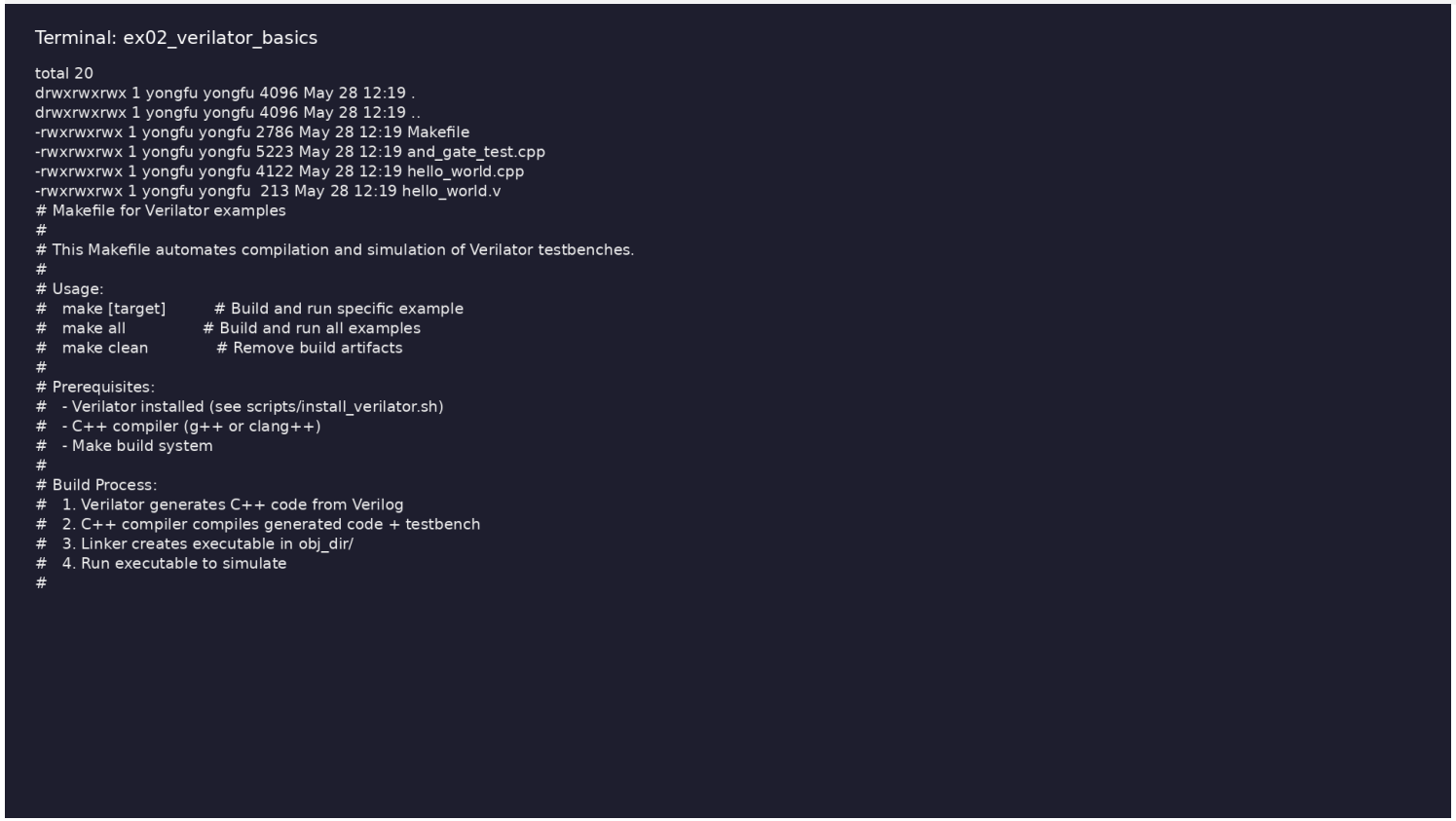
Demo: iverilog basics

```
$ cd module0/examples/iverilog_basics && make hello_world
```



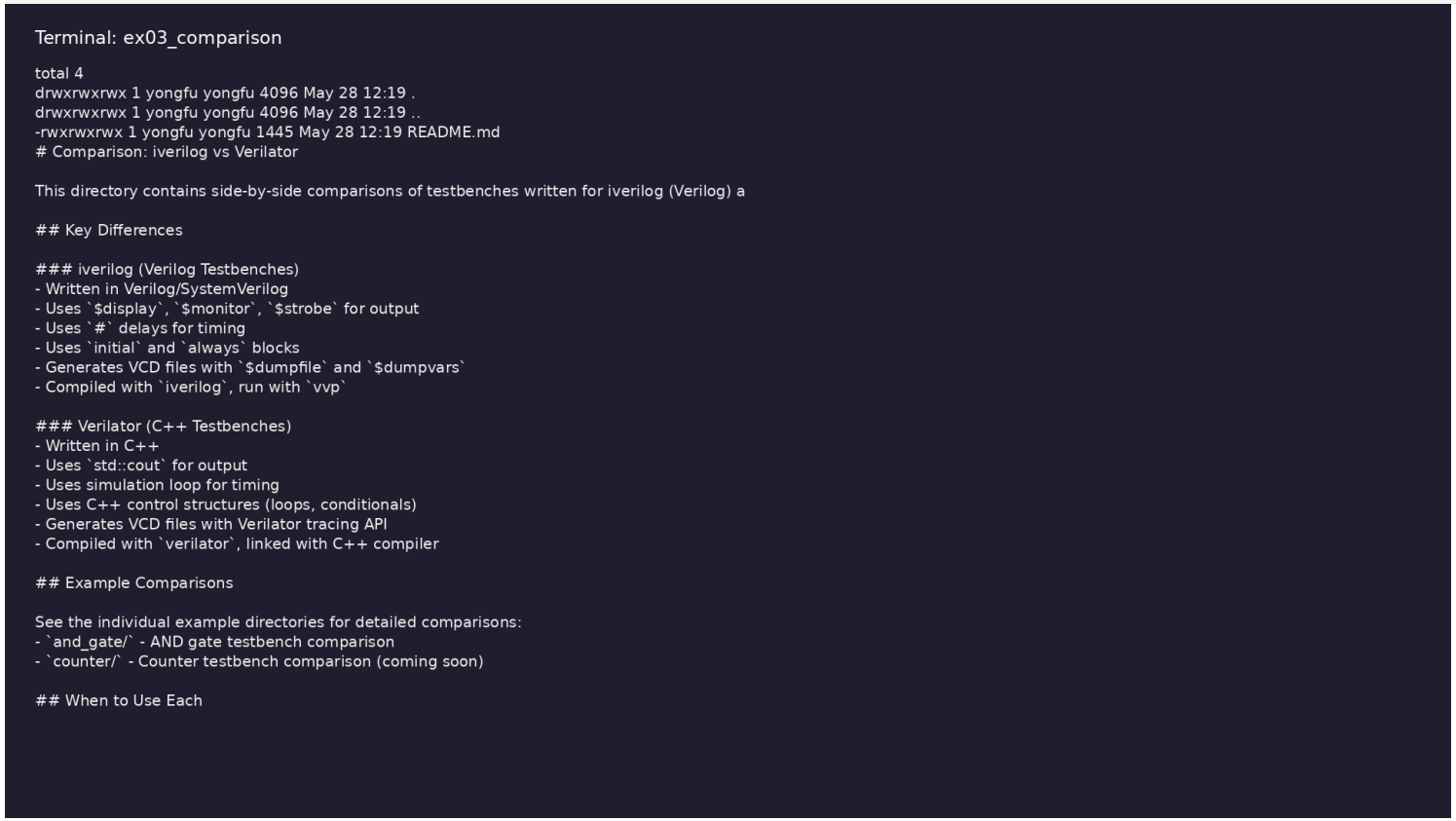
Demo: Verilator basics

```
$ ls -la module0/examples/verilator_basics && head -20
      module0/examples/verilator_basics/Makefile
```



Demo: Tool comparison

```
$ cd module0/examples/comparison && ls -la && head -30 README.md
      2>/dev/null || head -30 Makefile
```



Practice & assessment

What you should know (1/2)

- Install and configure all required tools
- Set up iverilog
- Set up Verilator
- Install and verify GTKWave
- Configure your IDE for verification work
- Create a basic project structure

What you should know (2/2)

- Run simple verification tests with both tools
- Understand tool differences and selection criteria
- Troubleshoot common installation issues

Exercises

- Installation Verification
- Environment Setup
- First Tests
- IDE Configuration
- Project Structure

Exercises

- Tool Comparison

Assessment checklist

- Can install all required tools independently
- Can set up iverilog
- Can verify iverilog installation
- Can set up Verilator
- Can verify Verilator installation
- Can install and verify GTKWave

Summary & next steps

- Set up verification environment with iverilog and Verilator
- Complete module0/CHECKLIST.md
- Review module0/EXAMPLES.md and run each lab
- Next: Next module in course